МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«Казанский национальный исследовательский технический университет им. А.Н. Туполева – КАИ»

Институт компьютерных технологий и защиты информации

Кафедра Систем Информационной Безопасности

Лабораторная работа № 1

Дисциплина: «Аппаратные средства телекоммуникационных систем»

Тема: «Основы OpenMP: параллельные области, распределение работ, синхронизация потоков»

Работу выполнил

Студент гр. 4404

Ионов И.А.

Казань, 2026

1. Цель работы

Освоить создание и запуск потоков, распределение итераций цикла между потоками (аналоги schedule static, dynamic, guided из OpenMP) и способы синхронизации, позволяющие задать порядок выполнения потоков.

2. Постановка задач

Задача 1. Написать программу, в которой каждый поток печатает свой идентификатор, общее количество потоков и строку «Hello World». Запустить с 8 потоками. Всегда ли вывод идентичен? Почему?

Задача 2. Объявить массив из 16000 элементов, a[i] = i. Построить массив b, в котором (кроме крайних элементов) b[i] = (a[i-1] + a[i] + a[i+1]) / 3.0. Запустить с 8 потоками при различных типах распределения работ.

Задача 3. Модифицировать задачу 1 так, чтобы потоки печатали идентификаторы в обратном порядке. Найти как можно больше способов.

3. Реализация

Программы написаны на Java без внешних библиотек; число потоков и размер задачи передаются через командную строку. Файлы: lab1/Task1.java, Task2.java, Task3.java.

Task1: создаются n объектов Thread (fork), каждый выводит строку, затем main ждёт их через join.

Task2: типы schedule реализованы вручную: static (один блок на поток), static с chunk (циклическая раздача блоков), dynamic (общий атомарный счётчик блоков), guided (размер блока уменьшается по мере исчерпания итераций). Результат каждого запуска сверяется с последовательной версией. Крайние элементы b[0] и b[n-1] не вычисляются и остаются нулевыми.

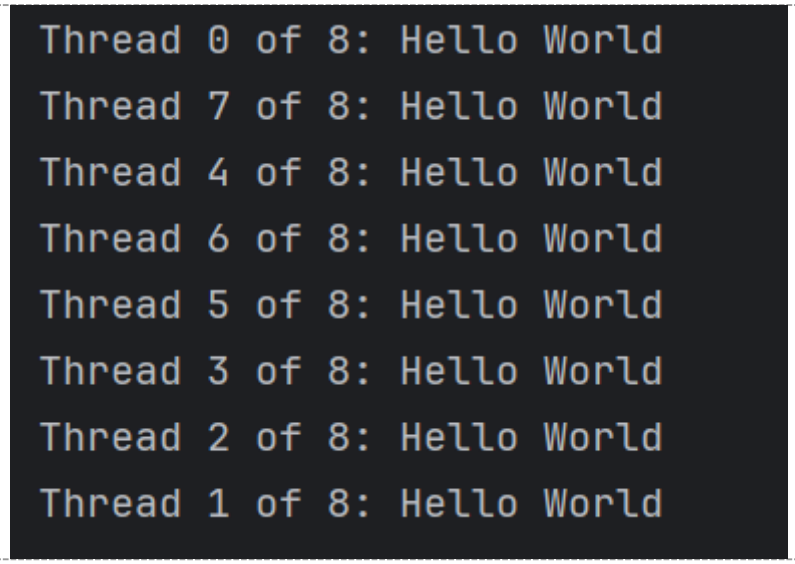
Task3: реализовано шесть способов (см. скриншоты 9–14): join-цепочка, монитор wait/notifyAll, семафоры, CountDownLatch, CyclicBarrier, сбор результатов в массив с выводом из main.

Сборка и запуск (подробнее в README папки lab1):

```
cd lab1
javac Task1.java Task2.java Task3.java
java Task1 8
java Task2 8 16000 dynamic 100
java Task3 8 5
```

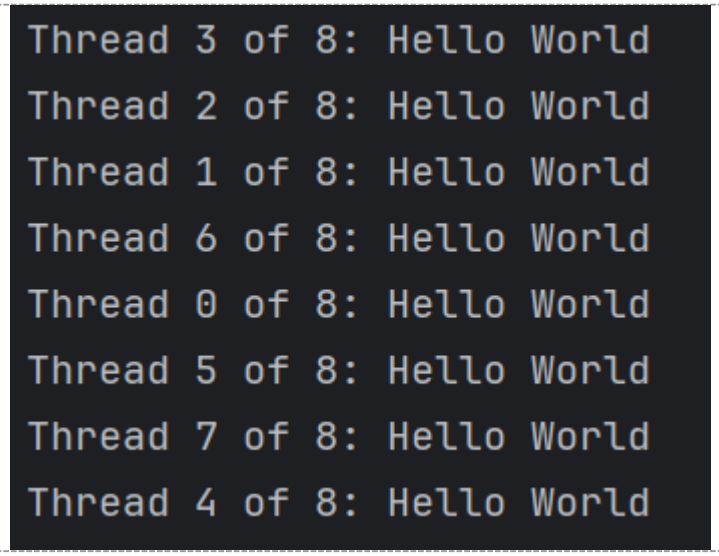
4. Ход работы и скриншоты

Скриншот 1. Запуск: `java Task1 8`



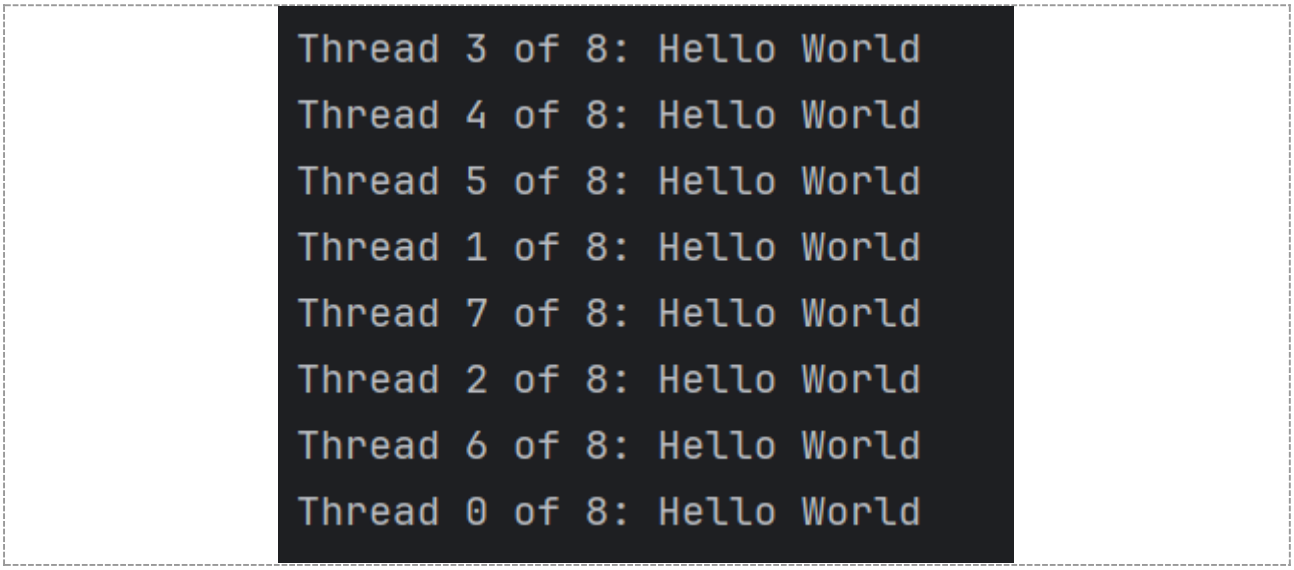
Пояснение. Первый запуск программы с 8 потоками. Каждый поток вывел свой номер, общее число потоков и Hello World; каждый номер от 0 до 7 встречается ровно один раз.

Скриншот 2. Запуск: `java Task1 8`



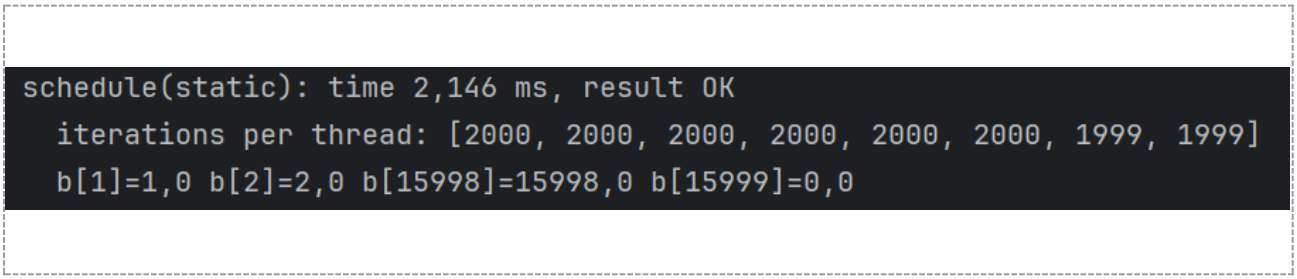
Пояснение. Второй запуск с теми же параметрами. Набор строк прежний, но порядок строк отличается от первого запуска.

Скриншот 3. Запуск: `java Task1 8`



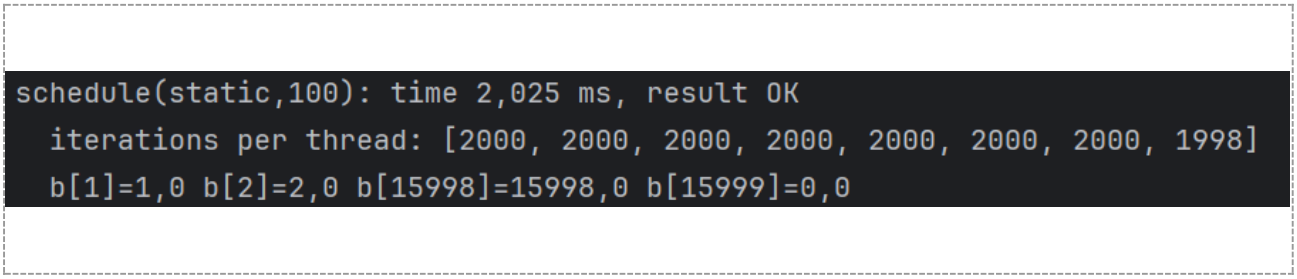
Пояснение. Третий запуск. Порядок вывода снова изменился: очерёдность обращения потоков к выводу не определена.

Скриншот 4. Запуск: `java Task2 8 16000 static`



Пояснение. schedule(static): 15998 итераций (элементы 1..n-2) делятся на 8 непрерывных блоков, каждый поток получает около 2000 итераций (число итераций по потокам выведено в строке «iterations per thread»). Накладные расходы минимальны, проверка с последовательной версией показывает «OK».

Скриншот 5. Запуск: `java Task2 8 16000 static 100`



Пояснение. schedule(static,100): блоки по 100 итераций раздаются потокам циклически (round-robin). Число итераций у потоков практически равно, но каждый поток обрабатывает не один отрезок, а набор отрезков.

Скриншот 6. Запуск: `java Task2 8 16000 dynamic 100`

```
schedule(dynamic,100): time 1,739 ms, result OK
  iterations per thread: [3200, 2798, 1800, 2400, 2600, 2300, 900, 0]
  b[1]=1,0 b[2]=2,0 b[15998]=15998,0 b[15999]=0,0
```

Пояснение. schedule(dynamic,100): блок забирает тот поток, который освободился. Так как итерация очень лёгкая, распределение между запуском и запуском различается: быстрый поток может забрать много блоков. Появляются небольшие накладные расходы на общий счётчик блоков.

Скриншот 7. Запуск: `java Task2 8 16000 guided 100`

```
schedule(guided,100): time 1,253 ms, result OK
  iterations per thread: [5840, 2350, 1439, 1172, 1026, 2360, 1178, 633]
  b[1]=1,0 b[2]=2,0 b[15998]=15998,0 b[15999]=0,0
```

Пояснение. schedule(guided,100): размер блока равен max(100, осталось/потоков), поэтому первые блоки большие, последние - по 100 итераций. Число обращений к общему счётчику меньше, чем у dynamic.

Скриншот 9. Запуск: `java Task3 8 1`

```
static void method1(int n) throws Exception {  1 usage
    for (int id = n - 1; id >= 0; id--) {
        final int fid = id;
        Thread t = new Thread(() -> System.out.println(msg(fid, n)));
        t.start();
        t.join();
    }
}
```

Пояснение. Способ 1: цепочка start/join: поток n-1 запускается первым и завершается до запуска следующего. Потоки выводят идентификаторы строго в порядке 7, 6, ..., 0.

Скриншот 10. Запуск: `java Task3 8 2`

```
static void method2(int n) throws Exception {  1 usage
    final Object lock = new Object();
    final int[] turn = {n - 1};
    Thread[] ts = new Thread[n];
    for (int i = 0; i < n; i++) {
        final int id = i;
        ts[i] = new Thread(() -> {
            synchronized (lock) {
                try {
                    while (turn[0] != id) lock.wait();
                } catch (InterruptedException e) { return; }
                System.out.println(msg(id, n));
                turn[0]--;
                lock.notifyAll();
            }
        });
    }
    for (Thread t : ts) t.start();
    for (Thread t : ts) t.join();
}
```

Пояснение. Способ 2: монитор: общая переменная turn и wait/notifyAll. Потоки выводят идентификаторы строго в порядке 7, 6, ..., 0.

Скриншот 11. Запуск: `java Task3 8 3`

```
static void method3(int n) throws Exception {  1 usage
    Semaphore[] sem = new Semaphore[n];
    for (int i = 0; i < n; i++) sem[i] = new Semaphore(i == n - 1 ? 1 : 0);
    Thread[] ts = new Thread[n];
    for (int i = 0; i < n; i++) {
        final int id = i;
        ts[i] = new Thread(() -> {
            try {
                sem[id].acquire();
                System.out.println(msg(id, n));
                if (id > 0) sem[id - 1].release();
            } catch (InterruptedException ignored) { }
        });
    }
    for (Thread t : ts) t.start();
    for (Thread t : ts) t.join();
}
```

Пояснение. Способ 3: семафоры: поток i ждёт свой семафор и после печати открывает семафор потока i-1. Потоки выводят идентификаторы строго в порядке 7, 6, ..., 0.

Скриншот 12. Запуск: `java Task3 8 4`

```
static void method4(int n) throws Exception {  1 usage
    CountDownLatch[] done = new CountDownLatch[n];
    for (int i = 0; i < n; i++) done[i] = new CountDownLatch(1);
    Thread[] ts = new Thread[n];
    for (int i = 0; i < n; i++) {
        final int id = i;
        ts[i] = new Thread(() -> {
            try {
                if (id < n - 1) done[id + 1].await();
                System.out.println(msg(id, n));
                done[id].countDown();
            } catch (InterruptedException ignored) { }
        });
    }
    for (Thread t : ts) t.start();
    for (Thread t : ts) t.join();
}
```

Пояснение. Способ 4: CountDownLatch: поток i ждёт завершения печати потоком i+1. Потоки выводят идентификаторы строго в порядке 7, 6, ..., 0.

Скриншот 13. Запуск: `java Task3 8 5`

```
static void method5(int n) throws Exception {  1 usage
    CyclicBarrier barrier = new CyclicBarrier(n);
    Thread[] ts = new Thread[n];
    for (int i = 0; i < n; i++) {
        final int id = i;
        ts[i] = new Thread(() -> {
            try {
                for (int phase = 0; phase < n; phase++) {
                    if (phase == n - 1 - id) System.out.println(msg(id, n));
                    barrier.await();      // все ждут, пока очередной поток напечатает
                }
            } catch (InterruptedException | BrokenBarrierException ignored) { }
        });
    }
    for (Thread t : ts) t.start();
    for (Thread t : ts) t.join();
}
```

Пояснение. Способ 5: CyclicBarrier: в каждой из n фаз печатает один поток, остальные ждут на барьере. Потоки выводят идентификаторы строго в порядке 7, 6, ..., 0.

Скриншот 14. Запуск: `java Task3 8 6`

```
static void method6(int n) throws Exception {  1 usage
    String[] res = new String[n];
    Thread[] ts = new Thread[n];
    for (int i = 0; i < n; i++) {
        final int id = i;
        ts[i] = new Thread(() -> res[id] = msg(id, n));
        ts[i].start();
    }
    for (Thread t : ts) t.join();
    for (int i = n - 1; i >= 0; i--) System.out.println(res[i]);
}
```

Пояснение. Способ 6: потоки пишут строки в общий массив, main выводит массив с конца. Потоки выводят идентификаторы строго в порядке 7, 6, ..., 0.

5. Ответы на вопросы

Вопрос. Всегда ли вывод программы из задачи 1 идентичен? Почему?

Ответ. Нет. Набор строк всегда один и тот же (каждый поток выводит свою строку ровно один раз), но порядок строк меняется от запуска к запуску. Потоки запускаются и планируются операционной системой независимо друг от друга, синхронизации между ними нет, поэтому порядок, в котором они доходят до оператора вывода, не определён. Это состояние гонки за общий ресурс (вывод). Отдельная строка при этом не разрывается, так как println синхронизирован.

Вопрос. Как влияет тип распределения работ в задаче 2?

Ответ. Результат (массив b) от типа распределения не зависит, так как итерации независимы. Меняется только то, какие итерации достаются каким потокам, и время работы. static даёт минимум накладных расходов и подходит при равной стоимости итераций; dynamic и guided балансируют загрузку при неравномерных итерациях ценой обращений к общему счётчику; guided уменьшает число таких обращений по сравнению с dynamic. На скриншоте с heavy=1 это видно по времени и по числу итераций на поток.

Вопрос. Какие способы обратного вывода найдены в задаче 3?

Ответ. Шесть: (1) join-цепочка, (2) монитор wait/notifyAll, (3) цепочка семафоров, (4) цепочка CountDownLatch, (5) CyclicBarrier с n фазами, (6) сбор результатов в массив и вывод из main.

В способах 2–5 потоки запущены в обычном порядке и сами ждут своей очереди; в способе 1 порядок задан порядком запуска, а параллелизма нет; в способе 6 печатает не поток, а главный поток.

6. Выводы

- Порядок выполнения и вывода потоков без синхронизации не определён, поэтому от него нельзя зависеть.
- Тип распределения итераций влияет на балансировку нагрузки и накладные расходы: static выгоден при равномерных итерациях, dynamic и guided - при неравномерных.
- Заданный порядок выполнения достигается явной синхронизацией (монитор, семафоры, барьер, латчи), но платой становится потеря параллелизма на упорядоченных участках.